

LAPORAN QUIZ PENGUJIAN PIRANTI LUNAK

Pengujian Modul Login & Register dengan Selenium dan CI/CD GitHub Actions

Field	Keterangan
Nama	Igan
NIM	(isi NIM Anda)
Mata Kuliah	Pengujian Piranti Lunak (PPL)
Repository Asal	https://github.com/hermanka/quiz-pengupil
Repository Pengumpulan	https://github.com/Darkside0908/Quiz_PPL
Tanggal	20 Juli 2026

1. Deskripsi Aplikasi yang Diuji

Aplikasi yang diuji adalah aplikasi web sederhana berbasis PHP dan MySQL yang terdiri dari dua modul: **login.php** (autentikasi user) dan **register.php** (pendaftaran user baru). Data user disimpan pada tabel `users` (kolom: `id`, `name`, `username`, `email`, `password`) dengan password di-hash menggunakan `password_hash()` (bcrypt). Setelah login atau register berhasil, aplikasi melakukan redirect ke `index.php`.

Hasil analisis kode menemukan bahwa file `index.php` tidak tersedia di repository, sehingga alur sukses login/register tidak dapat diverifikasi tanpa membuat **stub**. Selain itu password user bawaan database dump tidak diketahui (tersimpan sebagai hash), sehingga diperlukan **driver** untuk menyiapkan data uji.

Catatan penting: seluruh source code asli (`login.php`, `register.php`, `koneksi.php`, `style.css`) **tidak diubah sama sekali**. Semua pekerjaan pengujian hanya berupa penambahan file baru (`stub`, `driver`, `test script`, `workflow CI`) — dibuktikan dengan `git diff` terhadap commit awal yang menghasilkan output kosong untuk keempat file tersebut. Karena `koneksi.php` meng-hardcode kredensial (`host localhost`, `user root`, `password kosong`, `database quiz_pengupil`), environment pengujian (lokal maupun CI) yang dikonfigurasi untuk menyesuaikan diri terhadap kredensial tersebut, bukan sebaliknya.

2. Penggunaan Stub dan Driver

Komponen	File	Peran
Stub	<code>index.php</code>	Menggantikan halaman dashboard yang tidak ada di repository. Menjadi target redirect setelah login/register sukses, menampilkan elemen <code>#welcome</code> berisi username yang diverifikasi oleh test.
Stub	<code>logout.php</code>	Pendamping stub <code>index.php</code> untuk menghancurkan sesi, dipakai saat reset kondisi antar test.
Driver	<code>tests/seed.php</code>	Test driver yang menyiapkan test fixture: membuat user uji 'testuser' dengan password diketahui ('Password123!') langsung ke database. Menggunakan <code>require(koneksi.php)</code> asli tanpa modifikasi apa pun — hanya menambah data, bukan mengubah cara koneksi dibuat.
Driver	<code>tests/conftest.py</code>	Driver pytest: fixture Selenium WebDriver (Chrome headless), koneksi langsung ke MySQL (PyMySQL) dengan kredensial yang disamakan persis dengan hardcoded di <code>koneksi.php</code> , untuk verifikasi back-door dan pembersihan data uji, serta generator data user unik.

3. Rancangan Test Case

3.1 Modul Login (`login.php`)

ID	Skenario	Data Uji	Hasil yang Diharapkan
TC-L01	Login dengan kredensial valid	username: testuser password: Password123!	Redirect ke index.php, tampil pesan selamat datang berisi username
TC-L02	Login dengan password salah	username: testuser password: password_ngawur_999	Tidak masuk dashboard, tetap di login.php
TC-L03	Login dengan username tidak terdaftar	username: user_tidak_ada_xyz password: apapun123	Tidak masuk dashboard, tampil pesan error
TC-L04	Submit form kosong	username: (kosong) password: (kosong)	Tampil pesan 'Data tidak boleh kosong !!'
TC-L05	SQL Injection pada form login	username: ' OR '1'='1' -- password: ' OR '1'='1'	Login GAGAL, payload tidak boleh menembus autentikasi
TC-L06	Input hanya berisi spasi	username: (spasi) password: (spasi)	Dianggap kosong, tampil pesan 'Data tidak boleh kosong !!'

3.2 Modul Register (register.php)

ID	Skenario	Data Uji	Hasil yang Diharapkan
TC-R01	Register dengan semua field valid	Data user unik (username uji_<random>), password = re-password	Redirect ke index.php dan user tersimpan di database (diverifikasi via query langsung)
TC-R02	Password dan Re-Password berbeda	password: RahasiaUji123! repassword: RahasiaUji123!beda	Registrasi ditolak, tampil pesan 'Password tidak sama !!'
TC-R03	Username sudah terdaftar	username: testuser (sudah ada di DB)	Registrasi ditolak, tampil pesan 'Username sudah terdaftar !!'
TC-R04	Submit form kosong	Semua field kosong	Tampil pesan 'Data tidak boleh kosong !!'
TC-R05	End-to-end: register lalu login	Register user baru, hapus cookie sesi, lalu login dengan akun tersebut	Login berhasil, redirect ke index.php
TC-R06	Kolom name tersimpan dengan benar	Register user baru, cek kolom name via query DB	Kolom name = nama yang diinput. Test ini ditandai xfail (expected failure) karena ditemukan bug variabel \$nama

4. Script Selenium (Python + pytest)

Pengujian diimplementasikan dengan Selenium WebDriver 4 (Chrome headless), pytest sebagai test runner, dan PyMySQL untuk verifikasi data langsung ke database. Kode lengkap tersedia di folder tests/ pada repository. Berikut cuplikan utamanya.

tests/conftest.py (fixture driver)

```
@pytest.fixture
def driver():
    """Selenium WebDriver: Chrome headless, fresh session per test."""
    opts = Options()
    opts.add_argument("--headless=new")
    opts.add_argument("--no-sandbox")
    opts.add_argument("--disable-dev-shm-usage")
    drv = webdriver.Chrome(options=opts)
    drv.implicitly_wait(5)
    yield drv
    drv.quit()
```

tests/test_login.py (contoh test case)

```
def do_login(driver, username, password):
    driver.get(f"{BASE_URL}/login.php")
    driver.find_element(By.ID, "username").send_keys(username)
    driver.find_element(By.ID, "InputPassword").send_keys(password)
    driver.find_element(By.NAME, "submit").click()

def test_tc_l01_login_valid(self, driver):
    do_login(driver, SEED_USERNAME, SEED_PASSWORD)
    assert driver.current_url.endswith("index.php")
    welcome = driver.find_element(By.ID, "welcome").text
    assert SEED_USERNAME in welcome

def test_tc_l05_sql_injection(self, driver):
    do_login(driver, "' OR '1'='1' -- ", "' OR '1'='1'")
    assert "index.php" not in driver.current_url, \
        "CELAH KEAMANAN: SQL injection berhasil login!"
```

tests/test_register.py (contoh test case dengan verifikasi database)

```
def test_tc_r01_register_valid(self, driver, db, unique_user):
    u = unique_user
    do_register(driver, u["name"], u["email"], u["username"],
                u["password"], u["password"])
    assert driver.current_url.endswith("index.php")
    with db.cursor() as cur: # verifikasi back-door via driver DB
        cur.execute("SELECT username FROM users WHERE username = %s",
                    (u["username"],))
        assert cur.fetchone() is not None, "User tidak tersimpan di DB"

@pytest.mark.xfail(reason="Bug: INSERT memakai $nama yang tidak terdefinisi")
def test_tc_r06_kolom_name_tersimpan(self, driver, db, unique_user):
    ...
```

5. CI/CD Pipeline pada GitHub Actions

Pipeline didefinisikan pada `.github/workflows/selenium-tests.yml` dan berjalan otomatis pada setiap push dan pull request ke branch main. Tahapan pipeline:

Catatan desain penting: pipeline SENGAJA tidak memakai fitur "services" GitHub Actions untuk menjalankan MySQL sebagai container Docker terpisah. Jika memakai container terpisah, PHP harus terhubung lewat TCP ke host lain (mis. 127.0.0.1 dengan port mapping), sedangkan koneksi.php meng-hardcode host localhost yang oleh mysql diartikan sebagai koneksi lewat Unix socket lokal. Karena source code tidak boleh diubah, MySQL Server justru dipasang & dijalankan **langsung di dalam runner** (bukan sebagai container terpisah), lalu dikonfigurasi ulang (ALTER USER) agar user root memakai password kosong — persis meniru environment yang diasumsikan koneksi.php.

Tahap	Keterangan
1. Checkout	actions/checkout@v4 — mengambil source code apa adanya, tanpa modifikasi
2. Setup & konfigurasi MySQL	MySQL Server bawaan runner dijalankan langsung (bukan container), root diubah agar password kosong via mysql_native_password, database quiz_pengupil dibuat — menyesuaikan diri ke koneksi.php, bukan mengubah koneksi.php
3. Setup PHP 8.2	Menggunakan shivammathur/setup-php dengan ekstensi mysqli
4. Import skema DB	mysql -u root quiz_pengupil < db/quiz_pengupil.sql
5. Seeding (driver)	php tests/seed.php — memakai koneksi.php asli, membuat user uji dengan password diketahui
6. Jalankan aplikasi	PHP built-in server di port 8000. Flag -d output_buffering=4096 adalah konfigurasi runtime server saja (bukan perubahan source code), meniru perilaku default XAMPP
7. Setup Python + dependency	Python 3.12, pip install -r requirements.txt (Chrome sudah tersedia di runner ubuntu-latest, chromedriver diunduh otomatis oleh Selenium Manager)
8. Eksekusi test	pytest tests/ -v dengan laporan JUnit XML
9. Upload artifact	Hasil test (test-results.xml) diunggah sebagai artifact

Isi file workflow (.github/workflows/selenium-tests.yml)

```
name: Selenium Tests (Login & Register)
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
  workflow_dispatch:
permissions:
  contents: read
jobs:
  selenium-test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4

      # koneksi.php TIDAK diubah (host=localhost, user=root, password="").
      # MySQL dipasang & dikonfigurasi langsung di runner (bukan container
      # "services") agar koneksi via socket "localhost" tetap berfungsi.
      - name: Install & configure MySQL to match koneksi.php
        run: |
          sudo systemctl start mysql.service
          sudo mysql -e "ALTER USER 'root'@'localhost' \
            IDENTIFIED WITH mysql_native_password BY '';"
          sudo mysql -e "FLUSH PRIVILEGES;"
          sudo mysql -e "CREATE DATABASE IF NOT EXISTS quiz_pengupil;"

      - uses: shivammathur/setup-php@v2
        with: { php-version: "8.2", extensions: mysqli, tools: none }

      - run: mysql -u root quiz_pengupil < db/quiz_pengupil.sql
      - run: php tests/seed.php # driver, pakai koneksi.php asli

      - name: Start PHP built-in server
        run: |
          php -d output_buffering=4096 -S 127.0.0.1:8000 \
            > php-server.log 2>&1 &
```

```

sleep 2
curl -sf http://127.0.0.1:8000/login.php > /dev/null

- uses: actions/setup-python@v5
  with: { python-version: "3.12" }
- run: pip install -r requirements.txt
- run: pytest tests/ -v --tb=short --junitxml=test-results.xml

- if: always()
  uses: actions/upload-artifact@v4
  with: { name: test-results, path: test-results.xml }

```

6. Defect yang Ditemukan Selama Pengujian

No	Lokasi	Deskripsi Defect	Dampak
D-01	register.php (baris INSERT)	Query INSERT memakai variabel \$nama yang tidak pernah didefinisikan (variabel yang benar adalah \$name).	Kolom name selalu tersimpan kosong. Dibuktikan oleh TC-R06 (xfail) dan terlihat pada data dump bawaan yang kolom name-nya kosong.
D-02	login.php	Jika password salah, aplikasi tidak menampilkan pesan error apa pun (silent failure).	User tidak mendapat umpan balik; pengalaman pengguna buruk (TC-L02).
D-03	login.php	Username tidak terdaftar menampilkan pesan 'Register User Gagal !!' yang tidak sesuai konteks halaman login.	Pesan membingungkan user (TC-L03).
D-04	register.php	HTML dikirim ke browser sebelum header('Location: ...') dan session_start().	Redirect dan sesi hanya bekerja bila output_buffering aktif; pipeline menjalankan PHP dengan -d output_buffering=4096 agar perilaku sama seperti XAMPP.
D-05	register.php / login.php	Inkonsistensi kunci sesi: register memeriksa \$_SESSION['user '] tetapi yang di-set adalah \$_SESSION['username'].	Pemeriksaan 'sudah login' pada halaman register tidak pernah berfungsi.

7. Cara Menjalankan Pengujian

Lokal:

```

mysql -u root quiz_pengupil < db/quiz_pengupil.sql # import DB
php tests/seed.php # seeding user uji
php -d output_buffering=4096 -S 127.0.0.1:8000 # jalankan aplikasi
pip install -r requirements.txt
pytest tests/ -v

```

CI/CD: cukup push ke branch main; hasil eksekusi seluruh test case dapat dilihat pada tab **Actions** di repository GitHub. Ekspektasi hasil: 11 test PASS dan 1 test XFAIL (TC-R06, expected failure yang mendokumentasikan defect D-01).

8. Kesimpulan

Sebanyak 12 test case (6 login, 6 register) berhasil dirancang dan diotomasi menggunakan Selenium WebDriver, **tanpa mengubah source code asli** (login.php, register.php, koneksi.php, style.css) sama sekali. Stub (index.php, logout.php) diperlukan karena halaman tujuan redirect tidak tersedia, dan driver (seed.php, conftest.py) diperlukan untuk menyiapkan data uji serta verifikasi langsung ke database — keduanya adalah file baru, bukan modifikasi. Environment pengujian, baik lokal maupun CI/CD, dikonfigurasi agar menyesuaikan diri terhadap kredensial yang sudah di-hardcode pada koneksi.php. Seluruh pengujian terintegrasi dalam pipeline CI/CD GitHub Actions sehingga berjalan otomatis pada setiap perubahan kode. Pengujian juga berhasil menemukan 5 defect pada aplikasi, termasuk bug penyimpanan kolom name yang didokumentasikan sebagai expected failure tanpa diperbaiki.

Repository: https://github.com/Darkside0908/Quiz_PPL